



Чему научиться в этом году?

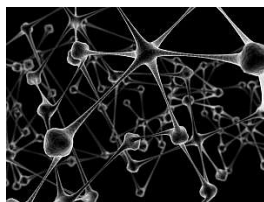
[Войти](#)**Dubrovkinmaxim**

14 янв 2010 в 02:42

Использование нейронной сети Хопфилда для решения простейшей задачи

9 мин 13K

Чулан



В настоящее время достаточно интересной областью программирования является использование нейронных сетей. Несмотря на всю перспективность этой идеи, большинство реализаций которые я встречал были связаны с различным анализом статистических выборок и предсказанием возможного значения какого-либо параметра.

Разговоры об искусственном интеллекте и громоздких экспертных системах — это конечно все хорошо, но как всю эту теорию приблизить к жизни, к нашим прикладным задачам?

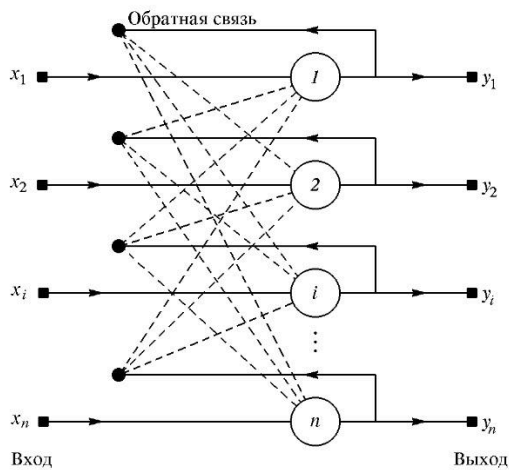
Немного о задаче

Требуется написать программу, которая в «зашумленной картинке» будет распознавать эталонные образы.

Выбор нейронной сети

В данной статье я не буду рассматривать теоретическую основу нейронных сетей, думаю данной информации в интернете достаточно=) Но все таки совсем без теоретического материала трудно будет понять про что я пишу.

И так для решения нашей задачи как нельзя лучше подходит нейронная сеть Хопфилда.



Она состоит из единственного слоя нейронов, число которых является одновременно числом входов и выходов сети. Каждый нейрон связан синапсами со всеми остальными нейронами, а также имеет один входной синапс, через который осуществляется ввод сигнала. Выходные сигналы, как обычно, образуются на аксонах.

Для данной сети на вход требуется некоторый набор двоичных сигналов, которые считаются образцовыми. Сеть должна уметь из произвольного неидеального сигнала, поданного на ее вход, выделить («вспомнить» по частичной информации) соответствующий образец (если такой есть) или в противном случае выдать наиболее похожий образ.

В общем случае, любой сигнал может быть описан вектором $X = \{x_i: i = 1, 2, \dots, n\}$, n — число нейронов в сети и размерность входных и выходных векторов. Каждый элемент x_i равен либо +1, либо -1.

Существует вектор, описывающий k -й образец. Когда сеть распознает (или «вспомнит») какой-либо образец на основе предъявленных ей данных, все выходы будут содержать именно его, т.е. Y вектор выходных значений сети: $Y = \{y_i: i = 1, 2, \dots, n\}$. В противном случае, если зашумление слишком сильное на выходах будет «мусор».

На стадии инициализации сети весовые коэффициенты синапсов устанавливаются следующим образом:

$$w_{ij} = \begin{cases} \sum_{k=1}^m x_i^k x_j^k, & i \neq j, \\ 0, & i = j. \end{cases}$$

Здесь i и j — индексы, соответственно, пресинаптического и постсинаптического нейронов;

Алгоритм функционирования сети следующий (t номер итерации):

1. На входы сети подается неизвестный сигнал. Фактически его ввод осуществляется непосредственной установкой значений:

$$y_i(0) = x_i, \quad i = 1, 2, \dots, n,$$

поэтому обозначение на схеме сети входных синапсов в явном виде носит чисто условный характер. Ноль в скобке справа от u_i означает нулевую итерацию в цикле работы сети.

2. Рассчитывается новое состояние нейронов

$$s_j(t+1) = \sum_{i=1}^n w_{ij} y_i(t), \quad j = 1, 2, \dots, n,$$

и новые значения аксонов

$$y_j(t+1) = f[s_j(t+1)],$$

3. Проверка, изменились ли выходные значения аксонов за последнюю итерацию. Если да переход к пункту 2, иначе (если выходы застabilizировались) — конец. При этом выходной вектор представляет собой образец, наилучшим образом сочетающийся с входными данными.

Таким образом, когда подается новый вектор, сеть переходит из вершины в вершину, пока не стабилизируется. Устойчивая вершина определяется сетевыми весами и текущими входами. Если из вершины в вершину, пока не стабилизируется. Устойчивая вершина определяется сетевыми весами и текущими входами. Если входной вектор частично неправилен или неполон, сеть стабилизируется в вершине, ближайшей к желаемой.

Доказано, что достаточным условием устойчивой работы такой сети является выполнение условий:

$$W_{ij} = W_{ji}, \quad W_{ii} = 0.$$

Как я уже говорил, иногда сеть не может провести распознавание и выдает на выходе несуществующий образ. Это связано с проблемой ограниченности возможностей сети. Для сети Хопфилда число запоминаемых образов N не должно превышать величины, примерно равной $0,15n$. Кроме того, если два образа A и B сильно похожи, они, возможно, будут вызывать у сети перекрестные ассоциации, т.е. предъявление на входы сети вектора A приведет к появлению на ее выходах вектора B и наоборот.

Приступим к написанию программы

И так для нейронной сети необходима бинарная последовательность (в нашем случае $-1/1$).

Пусть «1» — это черный цвет пикселя, а «-1» — это белый, таким образом сможем преобразовать картинку в последовательность.

Для быстроты и точности восстановления будем использовать следующую схему:

Картинка 100x100 пикселей и на каждый пиксель приходится свой нейрон.

Таким образом сеть будет состоять из 10 000 нейронов.

Начнем с представления элементарной единицы — нейрон:

```
//описание нейрона

private class Neuron

{

    //изменение состояние нейрона

    public void ChangeState()

    {

        if (s > 0) y = 1;

        if (s < 0) y = -1;

        if (s == 0) y = 0;

    }

    public int s;

    public int x; //вход

    public int y; //выход

    public int index;

}
```

* This source code was highlighted with Source Code Highlighter.

Теперь рассмотрим саму нейронную сеть, думаю для лучшего понимания следует привести основную часть листинга=)

```
namespace Neurons
{
    class NetHopfild
    {
        //описание нейрона

        private class Neuron
        {
            ...
        }

        private const int size = 10000;

        public NetHopfild()
        {
            //инстанцирование нейронов, мытрицы связей

            mass = new Neuron[size];

            for (int i = 0; i < size; i++)
            {
                mass[i] = new Neuron();
            }

            matrix_of_connect = new int[size, size];

            last_y = new int[size];
        }
    }
}
```

```
//

for (int i = 0; i < size; i++)

{

    mass[i].index = i;

}

for (int i = 0; i < size; i++)

{

    for (int j = 0; j < size; j++)

    {

        matrix_of_connect[i,j] = 0;

    }

}

}

public void Initialise(int [][]input)

{

    for (int i = 0; i < size; i++)

    {

        for (int j = 0; j < size; j++)

        {
```

```

        if (i == j) matrix_of_connect[i,j] = 0;

        else matrix_of_connect[i,j] += input[i] * input[j];

    }

}

}

```

```

public void FindImage(int []input)

```

```

{

```

```

    for (int i = 0; i < size; i++)    //вносим входящие значения

```

```

    {

```

```

        mass[i].x = input[i];

```

```

        last_y[i] = mass[i].y;

```

```

    }

```

```

    for (int k = 0; k < size; k++)    //сначала вычисляем S потом Y

```

```

    {

```

```

        mass[k].s = 0;

```

```

        for (int i = 0; i < size; i++)

```

```

        {

```

```

            mass[k].s = mass[k].s +

```

```

            matrix_of_connect[i,mass[k].index] * mass[i].x;

```

```
    }

    mass[k].ChangeState();

}

bool flag = true;

//проверяем на равенство входного и выходного векторов

for (int i = 0; i < size; i++)

{

    if(last_y[i]!=mass[i].y)

    {

        flag = false;

        break;

    }

}

if(flag == false)

{

    int[] temp = new int[size];

    for (int i = 0; i < size; i++)

    {

        temp[i] = mass[i].y;

    }

    //если неустойчивое состояние, то рекурсивный вызов
```

функции, пока не будет стабилизации сети

```
        FindImage(temp);

    }

}

private Neuron []mass;

private int[,] matrix_of_connect;

private int[] last_y;

    //индексатор для синапсов нейронов (выходные значения каждого
нейрона)

    public int [] Synapses

    {

        ...

    }

};

}
```

* This source code was highlighted with Source Code Highlighter.

И так сеть готова, теперь будем с ней работать.

В данном случае я не буду описывать весь исходник, опишу лишь основные моменты

Преобразование картинки в бинарную последовательность

```
private int [] BitmapToIntArray()

{

    //теперь перегоняем картинку в последовательность 1 и -1

    Color currentPixel;

    int[] temp = new int[size];

    int index = 0;

    for (int i = 0; i < picture.Height; i++)

    {

        for (int j = 0; j < picture.Width; j++)

        {

            currentPixel = picture.GetPixel(j, i);

            currentPixel.ToArgb();

            if (currentPixel == Color.FromArgb(255, 255, 255))

            {

                temp[index] = -1;

            }

            else

            {

                temp[index] = 1;

            }

            index++;

        }

    }

}
```

```
}  
  
}  
  
return temp;  
  
}
```

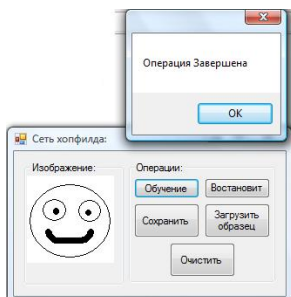
* This source code was highlighted with Source Code Highlighter.

Результаты

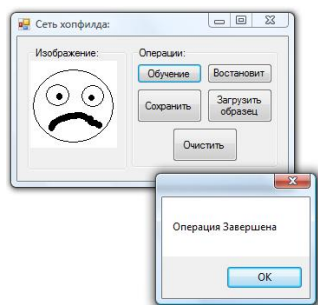
Пусть есть два образа-эталона

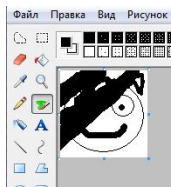
Обучаем нашу нейронную сеть



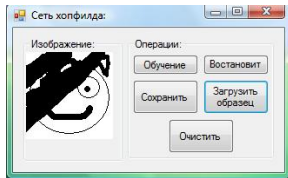
Аналогично обучаем второму образу.



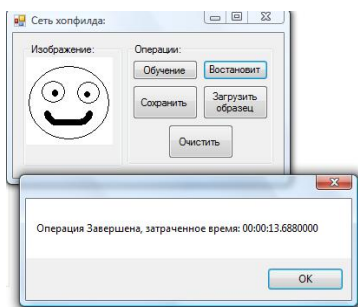
Создадим помехи в картинке:



Загружаем в программку искаженную картинку:

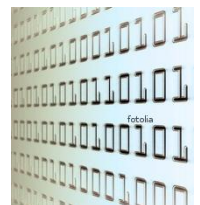


Жмем «Восстановить», через несколько секунд:



Как видим сеть располагая после обучения двумя смайликами выдала тот, на который искаженный рисунок больше всего подходит. Можно долго играть с процессом восстановления, но думаю в рамках этой статьи достаточно=) Экспериментируйте, получаются действительно интересные результаты ;)

А так как у нас сеть оперирует только с бинарными последовательностями, то думаю не составит труда переписать эту программку, для восстановления например печаток в словах=) и прочие вкусности=)



Для тех кому интересно выкладываю архив с образами и зашумлениями, сама программка

Ну и конечно если кто-то совсем проникнется идеей и понадобятся сорцы, напишите в комментариях, я обязательно выложу=)

UPD: Поправил страсти в коде=) На которые указали: @graninas, @Nagg, @Peregrinus

Теги: нейронная сеть

Хабы: Чулан

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



22

Карма

Максим @Dubrovkinmaxim

Senior .net developer

Подписаться



 Комментарии 28

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



Guren302

23 часа назад

Как ИИ убивает игровую индустрию (и не только её)



Простой



10 мин



14K

Обзор

+35

20

60



radiolok

4 часа назад

Стыдно когда видно! Советское видеонаблюдение



Простой



12 мин



4.5K

Обзор

+28

2

9



TrexSelectel

4 часа назад

Podman Compose: как устроен, что умеет, чем отличается от Docker Compose



5 мин



3.3K

+25

9

1



gtosss

9 часов назад

Найм в IT, 2026: 2 вакансии, 1 000 откликов в сутки



Простой



11 мин



11K

Мнение

+25

14

73



linux-over

18 часов назад

Строим VPN, устойчивый к SpyWare



Сложный



6 мин



21K

Аналитика

+25

109

89



Sonia_Black

18 часов назад

Мой CLAUDE.md — 582 строки. Вот зачем

 Средний  6 мин  22K

Кейс

 +25

 150

 35



RationalAnswer

7 часов назад

Приземление нутелловых космонавтов, а также деанон Сатоши Накамото

 9 мин  15K

Дайджест

 +23

 5

 15



morett1m

5 часов назад

Как Rust реализует трейт-объекты и почему dyn Trait медленнее дженериков

 11 мин  3.9K

Обзор

 +20

 5

 0



Catx2

22 часа назад

Трубы горят... и летят

 24 мин  13K

 +20

 15

 3



eliza_petyaeva

3 часа назад

Динамическое ценообразование в E-grocery. Раскладываем по полочкам

 Средний  11 мин  3.1K

Обзор

 +18

 12

 1

Результаты исследования: как айтишники относятся к ИИ

Турбо

Показать еще

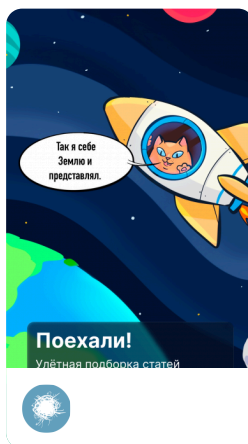
ИСТОРИИ



Годнота из блогов компаний



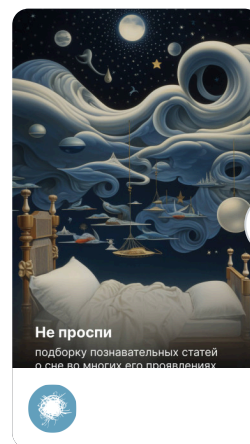
Хочу 450K



Вперёд к звёздам



Там на неведомых дорожках...



Самое время поспать

ВАКАНСИИ

Middle/Senior Data Scientist (команда ИИ и аналитика)

от 200 000 до 350 000 ₽ · Сбер · Москва

Системный администратор

до 180 000 ₽ · VINTEO · Краснодар

Frontend Developer (Vue.js) Middle / Senior

от 150 000 до 300 000 ₽ · UPBEAT SOFT · Можно удаленно

Больше вакансий на Хабр Карьере

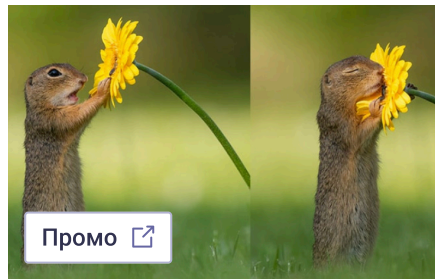
МИНУТОЧКУ ВНИМАНИЯ



Собери облачный пазл и
выиграй призы



Про IT на заводах: лучшие
статьи сезона Heavy Digital



Запахло весной и скидками в
Промокодусе

БЛИЖАЙШИЕ СОБЫТИЯ



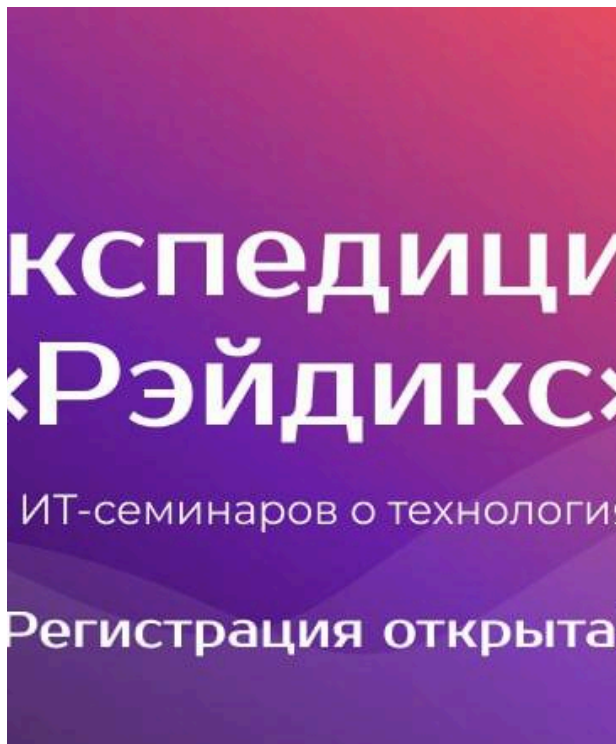
2 марта – 10 августа

**PROBA Awards —
международная
коммуникационная премия,
учрежденная в 2000 году**

Санкт-Петербург

[Маркетинг](#)

[Больше событий в календаре](#)



3 марта – 9 июня

Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск • Краснодар •
Минск • Калининград • Ростов-на-Дону

[Разработка](#) [Другое](#)

[Больше событий в календаре](#)



9 марта

**Всё,
старбрен**

Онлайн

[Марк](#)

[Больше](#)



 [Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr